

GUÍA EXPLICATIVA DEL PORTAL WEB MULTIMEDIA

Materia: Programación Web I

Estudiante: Fernando Santalla

Proyecto: Portal Web Multimedia con Maquetación Fluida Tradicional

Tema: Computación cuántica, inteligencia artificial y robótica

Fecha: 9 de septiembre de 2026

1. Introducción

El presente proyecto consiste en el desarrollo de un portal web multimedia basado en un episodio de podcast disponible en YouTube, relacionado con temas de tecnología, ciencia e innovación.

El episodio seleccionado es **“Computación cuántica, IA y robots: lo que viene”**, de Endor Podcast, con la participación de José Ignacio Latorre.

El objetivo principal fue integrar herramientas de inteligencia artificial generativa para realizar la curaduría del contenido y posteriormente desarrollar una página web utilizando HTML5 y CSS.

El proyecto fue realizado siguiendo una estrategia de **maquetación fluida tradicional**, sin utilizar Flexbox, CSS Grid, Media Queries ni JavaScript, de acuerdo con las restricciones establecidas en la actividad.

2. Curaduría del contenido mediante NotebookLM

Para la primera fase del proyecto se utilizó **NotebookLM** como herramienta de inteligencia artificial generativa.

Como fuente primaria se incorporó el enlace del episodio de YouTube. A partir del contenido analizado se obtuvieron los principales temas tratados, un resumen general, puntos importantes, citas relevantes y conclusiones.

Los principales ejes temáticos identificados fueron:

1. Computación cuántica y su importancia estratégica.
2. Fundamentos de la física cuántica.
3. Inteligencia Artificial General (AGI).
4. Ética y regulación de la inteligencia artificial.
5. Impacto socioeconómico de la automatización.

La información obtenida mediante NotebookLM fue utilizada posteriormente para organizar el contenido de las diferentes secciones del portal web.

3. Estructura semántica HTML5

La página fue estructurada utilizando etiquetas semánticas de HTML5 en lugar de utilizar únicamente elementos genéricos.

La utilización de elementos semánticos permite identificar mejor las diferentes regiones y tipos de contenido de una página web. Elementos como `header`, `nav`, `main` y `footer` también ayudan a que navegadores y tecnologías de asistencia puedan interpretar la estructura del sitio.

`<header>`

Se utilizó para contener la cabecera principal del portal.

En esta zona se presenta el título del proyecto y una introducción al contenido.

`<nav>`

Se utilizó para crear el menú de navegación principal.

Los enlaces permiten desplazarse directamente hacia las diferentes secciones del documento mediante identificadores (`id`).

`<main>`

Contiene el contenido principal de la página y agrupa las diferentes partes del portal.

`<section>`

Se utilizó para dividir el contenido en bloques temáticos.

Entre las secciones desarrolladas se encuentran el episodio, los temas principales, los puntos clave, la galería, las citas y las conclusiones.

`<article>`

Se utilizó para organizar contenidos independientes dentro de las diferentes secciones.

El elemento `article` está pensado para representar contenido que puede considerarse una composición independiente.

`<figure>`

Se utilizó para agrupar contenido multimedia, principalmente las imágenes de la galería y el contenido relacionado con el episodio.

`<figcaption>`

Se utilizó para colocar descripciones o títulos asociados a las imágenes y elementos multimedia.

`<footer>`

Se utilizó para crear el pie de página y colocar información complementaria del proyecto.

4. Contenido multimedia

Uno de los requisitos principales de la actividad era incorporar contenido multimedia.

Para cumplir este requisito se incorporó el episodio de YouTube mediante un elemento `<iframe>`.

El identificador utilizado para el video es:

IwaF7-COu1c

El reproductor se configuró para ocupar el espacio disponible dentro de su contenedor, evitando que el contenido sobrepase los límites establecidos.

También se incorporó una galería de imágenes relacionadas con los temas tratados en el episodio.

Las imágenes utilizan `width: 100%`, `max-width: 100%` y `height: auto`, permitiendo que mantengan sus proporciones al cambiar el tamaño disponible.

5. Estrategia de maquetación fluida tradicional

Uno de los requisitos más importantes del proyecto fue conseguir una página adaptable a diferentes tamaños de pantalla sin utilizar herramientas modernas de maquetación.

Por esta razón, el proyecto **no utiliza**:

- Flexbox.
- CSS Grid.
- Media Queries.
- JavaScript.
- Bootstrap.
- Tailwind.
- Frameworks o librerías CSS externas.

La adaptación se consiguió utilizando técnicas tradicionales de CSS.

5.1 Uso de porcentajes

Los elementos principales de la página utilizan un ancho relativo.

Por ejemplo:

```
width: 90%;
```

Esto significa que el elemento ocupa aproximadamente el 90% del espacio disponible de su contenedor.

A diferencia de utilizar un ancho fijo como `1200px`, el porcentaje permite que el elemento cambie proporcionalmente cuando cambia el tamaño de la ventana.

5.2 Uso de `max-width`

Además del porcentaje se utiliza:

```
max-width: 1200px;
```

Esto establece un límite máximo para el contenido.

La combinación:

```
width: 90%;  
max-width: 1200px;
```

permite que el contenido sea fluido en pantallas pequeñas y, al mismo tiempo, evita que se vuelva excesivamente ancho en pantallas grandes.

5.3 Uso de `inline-block`

La galería utiliza:

```
display: inline-block;
```

Esta propiedad permite colocar varios elementos uno al lado del otro sin utilizar Flexbox ni CSS Grid.

Los elementos de la galería tienen un ancho aproximado del 30%, permitiendo distribuir varias imágenes horizontalmente dentro del contenedor.

5.4 Adaptación de imágenes

Las imágenes utilizan:

```
width: 100%;  
max-width: 100%;  
height: auto;
```

El objetivo es que la imagen pueda reducir su tamaño cuando el espacio disponible sea menor, manteniendo su proporción.

5.5 Adaptación del video

El reproductor de YouTube utiliza:

```
width: 100%;  
max-width: 900px;
```

También se utiliza una altura basada en `vw`, junto con valores mínimos y máximos.

Esto permite que el video cambie proporcionalmente de tamaño dependiendo del ancho disponible.

6. Estrategia matemática utilizada

La adaptación de los elementos se basa principalmente en proporciones.

Por ejemplo, si un elemento utiliza:

```
width: 90%;
```

su ancho se calcula aproximadamente como:

Ancho del elemento = ancho disponible × 0,90

De manera similar, los elementos de la galería utilizan aproximadamente:

Ancho de cada elemento = ancho disponible × 0,30

Esto permite que los elementos cambien de tamaño de acuerdo con el espacio disponible.

Por ejemplo, si el contenedor tiene un ancho de 1000 píxeles:

1000 × 0,30 = 300 píxeles

Por lo tanto, cada elemento de la galería puede ocupar aproximadamente 300 píxeles, dejando espacio para márgenes y separación entre elementos.

Esta estrategia permite construir una interfaz fluida sin depender de Media Queries.

7. Diseño visual

Para el diseño visual se utilizó una paleta basada principalmente en tonos azules, blancos y grises.

El color azul oscuro se utiliza en la cabecera y el pie de página, mientras que los tonos azules más claros se utilizan para resaltar títulos, enlaces y elementos importantes.

El fondo claro permite mejorar la lectura del contenido.

También se utilizó una tipografía de la familia:

```
Arial, Helvetica, sans-serif;
```

La jerarquía visual se consiguió utilizando diferentes tamaños para títulos, subtítulos y texto normal.

8. Accesibilidad

Se incorporaron diferentes medidas básicas de accesibilidad.

Las imágenes cuentan con atributos `alt` para proporcionar una descripción alternativa cuando la imagen no puede visualizarse.

El menú utiliza enlaces internos para facilitar la navegación entre las diferentes partes de la página.

También se añadió una etiqueta descriptiva al elemento `<nav>`:

```
<nav aria-label="Navegación principal">
```

Esto permite identificar claramente la función del bloque de navegación.

La estructura semántica también contribuye a que las diferentes regiones de la página puedan ser identificadas por tecnologías de asistencia.

9. Organización de archivos

El proyecto mantiene una estructura organizada y modular:

```
mi-proyecto/  
├── index.html  
├── guia-explicativa.html  
├── css/  
│   └── styles.css  
└── img/  
    ├── imagen1.jpg  
    ├── imagen2.jpg  
    └── imagen3.jpg
```

El archivo `index.html` contiene la estructura y el contenido de la página.

El archivo `styles.css` contiene los estilos visuales y las reglas utilizadas para conseguir la maquetación fluida.

La carpeta `img` contiene las imágenes utilizadas en la galería y el contenido multimedia.

10. Restricciones técnicas cumplidas

Durante el desarrollo se respetaron las restricciones indicadas en la actividad.

No se utilizó:

- `display: flex`
- `display: grid`
- `@media`
- JavaScript
- Bootstrap
- Tailwind
- Frameworks CSS externos

Sí se utilizaron:

- HTML5 semántico.
 - Porcentajes (%).
 - `display: inline-block` .
 - `max-width` .
 - `width: 100%` .
 - `height: auto` .
 - Elementos multimedia mediante `<iframe>` .
 - Galería de imágenes.
 - Navegación interna.
 - CSS puro.
-

11. Conclusión

El desarrollo de este proyecto permitió integrar herramientas de inteligencia artificial generativa con conocimientos de desarrollo web.

NotebookLM permitió realizar la curaduría y síntesis del contenido del episodio seleccionado, facilitando la identificación de sus principales temas y puntos relevantes.

Posteriormente, mediante HTML5 se construyó una estructura semántica organizada y mediante CSS se desarrolló una interfaz visual y fluida.

La principal dificultad técnica fue conseguir la adaptación a diferentes tamaños de pantalla sin utilizar Flexbox, CSS Grid ni Media Queries. Para resolverlo se utilizaron porcentajes, `inline-block`, `max-width`, `width: 100%` y otras propiedades de CSS tradicional.

De esta manera, el proyecto cumple con los principales requisitos técnicos establecidos para el desafío y demuestra la aplicación práctica de HTML5 semántico, CSS y herramientas de inteligencia artificial generativa.